Persistence-only live inference

Implemented and regression-tested. A genuine live forecast remains unavailable.

Scope: a separate backend and dashboard inference path using the exact selected Persistence baseline rule. This supplement preserves the Phase 1-5 artifacts, original Phase 6 report, SQLite history and frozen 37-feature pipeline.

Evidence captured at 2026-11-11T19:45:58.637098+00:00 (UTC).

Method and strict acceptance conditions

Prediction: occupied(t + 15 minutes) = occupied(t). No fitting, model substitution, feature padding or synthetic fallback is performed. Automated tests establish equivalence with the existing PersistenceRegressor for all totals 0-12. This parameter-free adapter does not load the trained bundle or call the frozen ParkingPredictor interface.

A single backend SQLite snapshot must contain exactly P1-P12 with correct device assignments, known states, valid 2-400 cm distances, and observed=true. Backend stale flags block inference. Receipt and measurement timestamps must both be timezone-aware and at most min(configured freshness, 30 seconds) old; clocks more than two seconds ahead are rejected. Missing or unsynchronized measurement clocks block inference. Sequential stale board readings are never combined.

Endpoint and interface

GET /predictions/persistence-live reads one snapshot in a synchronous FastAPI route, keeping database I/O off the async event loop. The pure adapter writes no data. Responses identify inference_path=persistence_only_live, model=persistence, horizon_minutes=15 and data_source=api_sqlite_telemetry. They include evaluation time, freshness limit, the snapshot and per-slot blockers. Available responses contain equal current/predicted occupied counts and exact origin/target timestamps. Unavailable responses contain null counts and forecast timestamps.

Dashboard integration

The existing live Prediction & Analytics section displays the separately labelled Persistence-only path, its complete current count, 15-minute forecast, origin, target, last successful prediction and freshness. With missing/stale/UNKNOWN data it shows reasons and no forecast. Display-time revalidation prevents cached successful responses from becoming stale forecasts. Existing full ML controls, historical graphs, model comparison and synthetic retrospective results remain intact. Automatic and manual refresh reuse the existing live fragment.

Evidence and acceptance

Check	Actual result
Backend / dashboard regressions	PASS: 51 + 44 tests; includes 12 focused adapter/UI tests.
ML / circuit static regressions	PASS: 42 + 3 tests. Total: 140, all exit code 0.
Running API8001 / dashboard8503	PASS: HTTP 200 for /health and /_stcore/health.
Actual database inference	PASS (safe blocking): HTTP200, available=false, 12 stale-slot blockers; all forecast values null.
Actual API Streamlit AppTest	PASS: zero exceptions; 12 cards, occupied0/vacant0/unknown12; manual refresh refetched endpoint.
Running8503 WebSocket refresh	PASS: initial script success; 10-second timer registration; client-equivalent timed fragment rerun succeeded and advanced freshness time.
Visual browser rendering	BLOCKED: Computer Use could not establish the browser URL sufficiently to enforce policy. No bypass attempted.
Genuine fresh live prediction	NOT DEMONSTRATED: all current database observations stale. Success cases use explicitly isolated test fixtures.

The WebSocket check uses installed Streamlit protobuf messages and a normal WebSocket connection; it verifies the server and fragment protocol, not browser rendering or an independently observed browser timer. The client-equivalent rerun completed with status3 (fragment success); the initial script completed with status0. AppTest independently exercises UI elements and real HTTP API calls.

Preservation evidence

Before/after live SQLite file hashes are identical. Counts remain 563 telemetry messages, 2,252 historical observations and twelve current slots. No records, schema or original collector process were changed. Baseline comparison found no tracked changes under ml/, esp32/, backend/db.py, backend/schemas.py, dashboard/data_views.py or dashboard/prediction.py. Existing PDFs are unchanged; the full ML code block in dashboard/app.py is byte-equivalent after text decoding.

SQLite SHA-256: 1811930e8212c9bfd7df2d72533da593f3fb45ab2782f35f4dd0f4b6a5dcabe0

Frozen model SHA-256: cfc2adc80cc949e3c097530bca500079b857eee8b931be1e6d7de522f50335e1

Test boundaries

No new simulator sessions or C++ host executables were run during this closure. The earlier Windows Application Control restriction remains respected. The 140-test result covers the Python regression suites, including frozen artifact reload tests and static circuit checks; it is not a new Wokwi firmware runtime certification. Successful mocked/isolated forecasts are not genuine telemetry evidence.

Operation and remaining blockers

Verified local stack

Dashboard: http://127.0.0.1:8503. API: http://127.0.0.1:8001. The verification API uses the existing local SQLite database with MQTT ingestion disabled, avoiding a second collector. The original API8000 and Mosquitto were left running. Its MQTT-disabled health response does not establish broker connectivity. API_BASE_URL points the verified dashboard to8001; the existing full model adapter remains subprocess with ML_PYTHON set to the already approved isolated Windows runtime.

The original8000 process has not been restarted to load the new route. A later approved normal restart is required before dashboards pointing to8000 can use this endpoint. Do not interpret a404 on that old process as a freshness failure.

Reproducible commands (existing environments only)

Command	Purpose
.venv\Scripts\python.exe -B -m unittest discover -s backend/tests -v	51 backend tests; isolated scratch databases.
.venv\Scripts\python.exe -B -m unittest discover -s dashboard/tests -v	44 dashboard tests. Set PHASE5_TEST_ML_PYTHON to the approved isolated interpreter for native adapter tests.
%LOCALAPPDATA%\SmartParking\ml-runtime\Scripts\python.exe -B -m unittest discover -s ml/tests -v	42 ML tests using the approved runtime.
.venv\Scripts\python.exe -B esp32/tests/test_circuits.py	3 static circuit tests; no firmware changes.
GET http://127.0.0.1:8001/predictions/persistence-live	Read actual snapshot, null forecast and precise blocking reasons.

Shortest legitimate demonstration

All three boards must deliver genuine, known, timestamped observations for all twelve slots within the same30-second freshness window. Then the next refresh may display the actual complete occupancy as the15-minute Persistence forecast. Current slow sequential simulator operation has not achieved this overlap; no freshness relaxation, synthetic data or installation is authorized. Until overlap is observed, retain the unavailable state. The original full37-feature pipeline independently requires10,081 complete minute rows (seven days plus origin).

Remaining acceptance items

Pending: genuine twelve-slot fresh telemetry and a resulting live forecast; supported visual browser review; normal activation on the original API process when authorized. **Deferred:** cloud deployment, Linux runtime certification and sustained simultaneous simulator operation. No cloud resources, credentials, installations, paid services or security-policy changes were used. This closure commits locally; no remote push is claimed.

Evidence: reports/evidence/phase6_persistence_live_integration.json stores full test output, actual API responses, real-API AppTest checks, WebSocket refresh records, preservation hashes and tested source hashes. Render this supplement with the bundled ReportLab Python runtime using reports/generate_phase6_persistence_supplement.py; it refuses to overwrite existing PDFs.